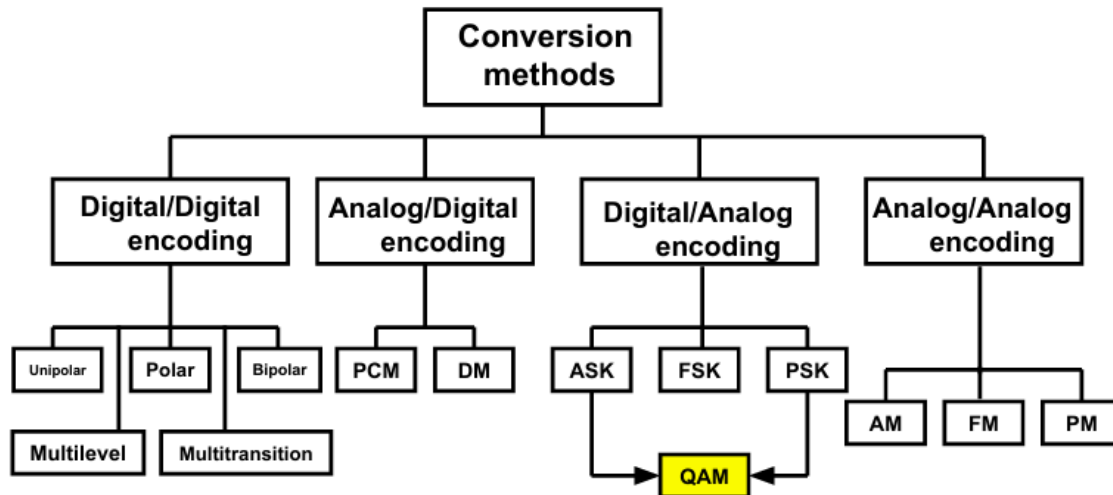


Digital transmissions

Digital Transmission

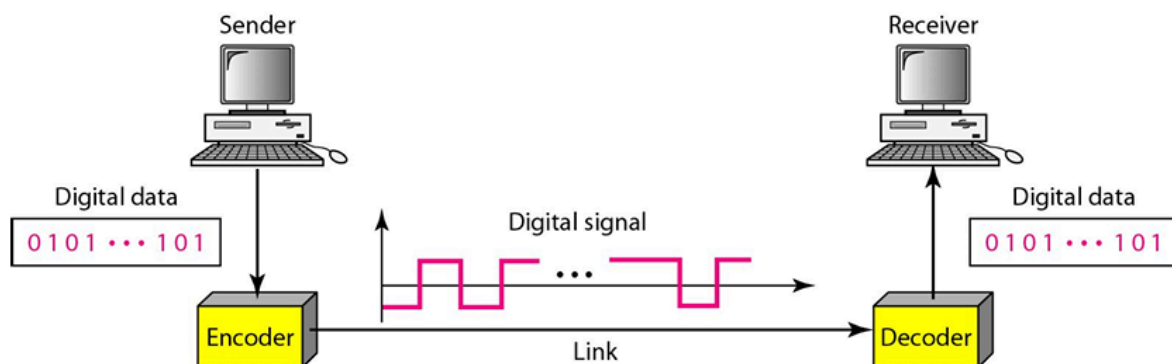
- ❑ The Information needs to be converted to either a digital signal or an analog signal for transmission



Digital-to-Digital Conversion

- ❑ Converts sequence of bits to a digital signal

Figure 4.1 Line coding and decoding



◆ Data Element

- The **smallest piece of information** to be transmitted.
- In digital transmission, this is simply a **bit** (0 or 1).
- Think of it as the *content* we want to send.

◆ Signal Element

- The **physical representation** of the data element on the medium (wire, fiber, etc.).
- It is the **shortest unit of the transmitted digital signal** (e.g., one voltage pulse, one phase shift, one frequency burst).
- Think of it as the *carrier* that transports the data element.

◆ Relationship

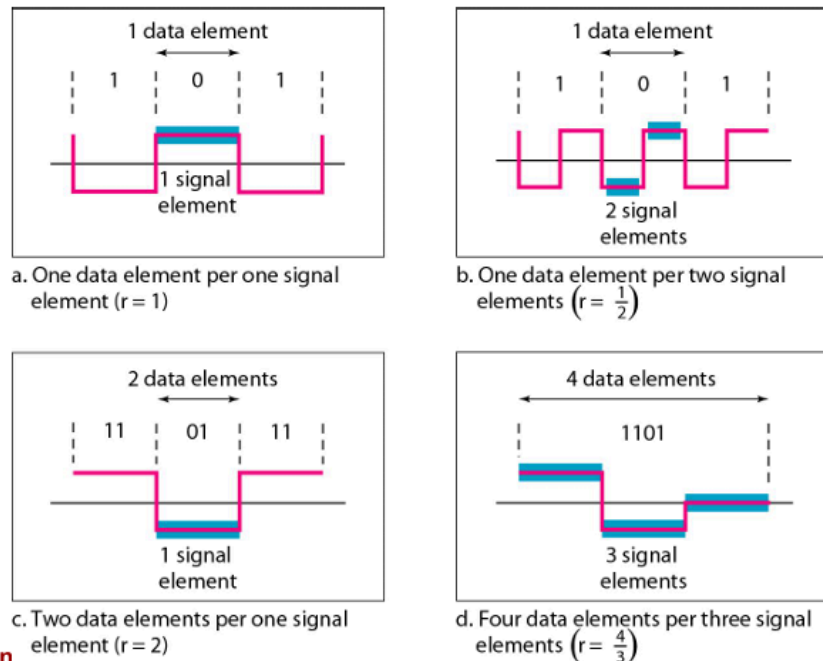
- **Data elements are being carried; signal elements are the carriers.**
- A data element (bit) is abstract, but it needs a signal element to actually travel across the medium.
- Depending on the line coding scheme:
 - 1 data element may map to **1 signal element** (e.g., NRZ).
 - 1 data element may require **multiple signal elements** (e.g., Manchester).
 - Or 1 signal element may carry **multiple data elements** (e.g., multilevel coding).

✓ In short

- **Data element = Information (bit)**
- **Signal element = Physical waveform chunk that carries it**

Data element versus Signal element

Figure 4.2 *Signal element versus data element*



Perfect — you've shared the figure “Data element versus Signal element” (Fig. 4.2). Let me explain it step by step:

◆ Key Concepts

- **Data element:** The *bit* (0 or 1). It's the smallest piece of information.
- **Signal element:** The physical change in the transmitted waveform (voltage, light, etc.) that carries one or more bits.

👉 The ratio between them is shown as $r = (\text{number of data elements}) / (\text{number of signal elements})$.

◆ Explanation of the Subfigures

(a) One data element per one signal element ($r = 1$)

- Each bit maps directly to one signal element.
- Example: Bit 1 = high voltage, Bit 0 = low voltage.
- Very simple mapping, used in **NRZ-L**, **NRZ-I** type codes.
- $r = 1 \rightarrow 1$ bit per 1 signal element.

(b) One data element per two signal elements ($r = 1/2$)

- Each data bit needs *two* signal elements to represent it.
- Example: In **Manchester coding**, each bit is represented by a transition (two halves).
- This makes synchronization easier, but the signal rate doubles compared to bit rate.
- $r = 1/2 \rightarrow 1$ bit spread across 2 signal elements.

(c) Two data elements per one signal element ($r = 2$)

- Here one signal element carries **2 bits**.
- Example: If the signal has 4 distinct voltage levels, each level can represent 00, 01, 10, or 11.
- This is **multilevel coding (M-ary line coding)**.
- $r = 2 \rightarrow$ compression, bandwidth-efficient but requires better noise performance.

(d) Four data elements per three signal elements ($r = 4/3$)

- A hybrid case where 3 signal elements carry 4 bits.

- Achieved using **block coding** (e.g., 4B/3T → 4 bits mapped to 3 ternary symbols).
- Used to balance efficiency and synchronization.
- $r = 4/3$ → each signal element carries more than 1 bit on average.

◆ Why is this important?

- This ratio r helps calculate the **signal rate (baud rate)** vs. **bit rate**.
- Formula:

$$S = c \times N \times r$$
 where
 - S = signal rate (baud),
 - N = data rate (bps),
 - r = data elements per signal element,
 - c = code-dependent factor.

✓ Summary:

- **(a)** Simple 1:1 mapping (NRZ codes).
 - **(b)** Two signals for one bit (Manchester).
 - **(c)** One signal carries 2 bits (Multilevel coding).
 - **(d)** Block coding efficiency (e.g., 4B/3T).
-

“Data Rate versus Signal Rate”.

◆ Key Terms

1. Data Rate (bit rate, N)

- Number of **data elements (bits)** transmitted per second.
 - Unit: **bps (bits per second)**.
👉 Example: If you transmit 1,000 bits in one second → data rate = 1 kbps.
-

2. Signal Rate (baud rate, S)

- Number of **signal elements (waveform changes)** transmitted per second.
 - Unit: **baud (symbols per second)**.
👉 Example: If you send 500 distinct signal pulses in one second → signal rate = 500 baud.
-

3. Relationship Between Them

$$S = c \times N \times \frac{1}{r} \quad (\text{baud})$$

Where:

- S = signal rate (baud)
- N = data rate (bps)
- r = number of data elements per signal element
- c = case factor (depends on code efficiency, usually between 0 and 1)

◆ Intuition

- Sometimes 1 signal element carries exactly 1 bit $\rightarrow$ then $r = 1$.
 - Example: NRZ line coding $\rightarrow$ baud rate $\approx$ bit rate.
- Sometimes 1 signal element carries multiple bits (multilevel coding).
 - Example: If 1 signal element has 4 levels, it can carry 2 bits $\rightarrow r = 2$. Baud rate becomes half the bit rate.
- Sometimes 1 bit requires multiple signal elements (Manchester coding).
 - Example: Each bit uses 2 signal elements $\rightarrow r = \frac{1}{2}$. Baud rate is **double** the bit rate.

○

◆ Example Calculations

Example 1: NRZ-I ($r = 1$, $c \approx 1/2$)

- Data rate = 1 Mbps
- Then $S' = \frac{1}{2} \times 1,000,000 \times \frac{1}{1} = 500,000$ baud
- So we need 500 kHz bandwidth minimum.

Example 2: Manchester ($r = \frac{1}{2}$, $c = 1$)

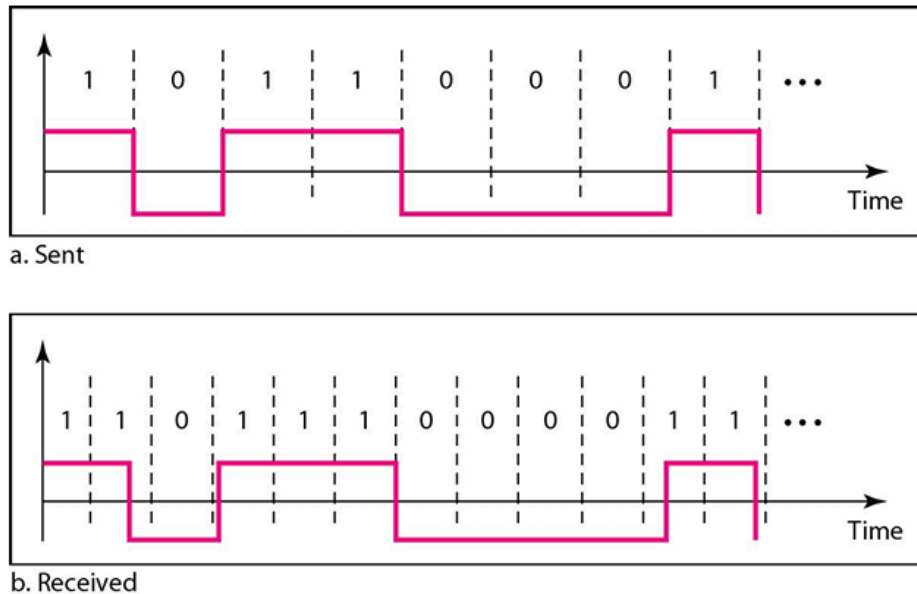
- Data rate = 1 Mbps
- Then $S' = 1 \times 1,000,000 \times \frac{1}{0.5} = 2,000,000$ baud
- So baud rate = 2 MHz, double the bit rate.

Example 3: Multilevel coding with 4 levels ($r = 2$, $c \approx 1$)

- Data rate = 1 Mbps
- Then $S' = 1 \times 1,000,000 \times \frac{1}{2} = 500,000$ baud
- So baud rate is half the bit rate.

Self-Synchronization

Figure 4.3 *Effect of lack of synchronization*



SELF-SYNCHRONIZATION

◆ Definition

Self-synchronization is the property of a digital signal that allows the **receiver's clock** to stay aligned with the **sender's clock** without needing a separate timing signal.

- The receiver's **bit intervals** must match the sender's.
- A **self-synchronizing signal** embeds timing information in the waveform (e.g., regular transitions).
- If the receiver's clock drifts, the built-in transitions act as "reset points" to bring it back in sync.

◆ Why It's Important

- In real systems, sender and receiver clocks are never perfectly matched.
 - Even a small drift (e.g., 0.1%) causes the receiver to **read extra or miss bits** over time.
 - Without synchronization, transmitted data gets misaligned → **bit errors**.
-

◆ Example with Figures

(a) Sent Signal

- Sequence of bits: 10110001...
- Transmitted correctly with clear boundaries between bits.

(b) Received Signal (without sync)

- Receiver clock drifts → it interprets the waveform at slightly wrong positions.
 - Result: Wrong bit sequence is recovered (11010001...).
 - This is the **effect of lack of synchronization**.
-

◆ How Self-Synchronization is Achieved

- **Line coding schemes with frequent transitions:**
 - **Manchester** and **Differential Manchester**: always have a mid-bit transition.
 - **RZ (Return-to-Zero)**: returns to zero every bit, adding timing info.
 - **Scrambling or block coding**: ensures no long runs of identical bits.
-

Self-Synchronization

Example 4.3

In a digital transmission, the receiver clock is 0.1 percent faster than the sender clock. How many extra bits per second does the receiver receive if the data rate is 1 kbps? How many if the data rate is 1 Mbps?

Solution

At 1 kbps, the receiver receives 1001 bps instead of 1000 bps.

1000 bits sent	1001 bits received	1 extra bps
----------------	--------------------	-------------

At 1 Mbps, the receiver receives 1,001,000 bps instead of 1,000,000 bps.

1,000,000 bits sent	1,001,000 bits received	1000 extra bps
---------------------	-------------------------	----------------



SELF-SYNCHRONIZATION (EXAMPLE 4.3)

◆ Problem Restated

In a digital transmission system, the **receiver's clock** runs **0.1% faster** than the **sender's clock**.

We want to know:

1. How many extra bits per second the receiver will interpret if the data rate is **1 kbps**.
2. How many extra bits per second the receiver will interpret if the data rate is **1 Mbps**.

◆ Step-by-Step Solution

Step 1: Understand the error

- A **0.1% faster clock** means the receiver samples slightly more often than it should.
- So it **reads more bits than the sender actually sends**.

Step 2: Case 1 → 1 kbps

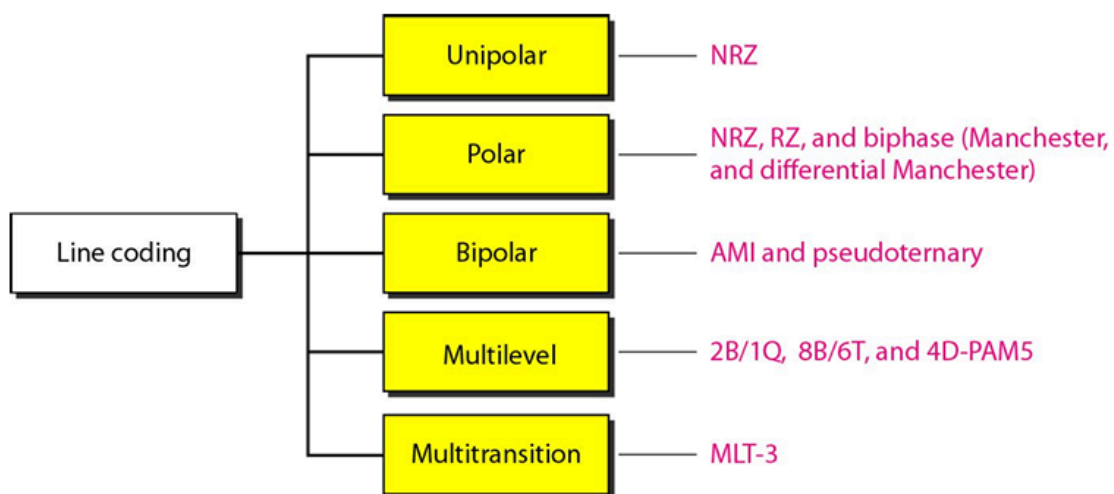
- Sender sends: **1000 bits per second**.
- Receiver is 0.1% faster → it thinks it should read $1000 \times (1 + 0.001) = 1001$ bits per second.
- Extra bits = $1001 - 1000 = 1$ bit/sec.

✓ At **1 kbps**, the receiver gets **1 extra bit each second**.

Step 3: Case 2 → 1 Mbps

- Sender sends: **1,000,000 bits per second**.
- Receiver is 0.1% faster → it thinks it should read $1,000,000 \times (1 + 0.001) = 1,001,000$ bits per second.
- Extra bits = $1,001,000 - 1,000,000 = 1000$ bits/sec.

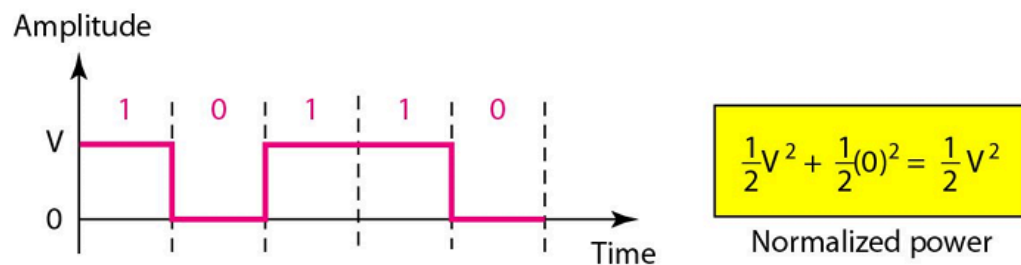
✓ At **1 Mbps**, the receiver gets **1000 extra bits each second**.



Line Coding Schemes

Figure 4.5 *Unipolar NRZ scheme*

□ Unipolar encoding uses only one voltage level.



UNIPOLAR NRZ LINE CODING

◆ What It Is

- Unipolar NRZ (Non-Return-to-Zero) is the **simplest line coding scheme**.
- It uses **only one non-zero voltage level** to represent data bits.

👉 In this scheme:

- **Bit 1** → Voltage = **+V**
- **Bit 0** → Voltage = **0 V**

So the signal only has **two levels: 0 and +V**.

◆ Waveform (Figure Explanation)

- The figure shows the data sequence: 1 0 1 1 0 1 0.
- Each 1 is mapped to a high voltage (+V).
- Each 0 is mapped to zero voltage.
- Unlike **RZ (Return-to-Zero)**, the signal **does not return to zero in the middle of the bit** → hence the name **Non-Return-to-Zero (NRZ)**.

◆ Characteristics

- **Simplicity:** Very easy to implement.
- **Bandwidth Efficiency:** Baud rate $\approx$ Bit rate (1 signal element per bit).
- **Synchronization:** Poor — if there are long runs of 0s or 1s, there are no transitions to help the receiver's clock.
- **DC Component:** Strong DC (average signal level is above 0).
- **Power Efficiency:** Not great compared to polar/bipolar codes.

◆ Advantages

- ✓ Simple and low-cost implementation.
- ✓ Useful in some optical systems (light ON = 1, OFF = 0).

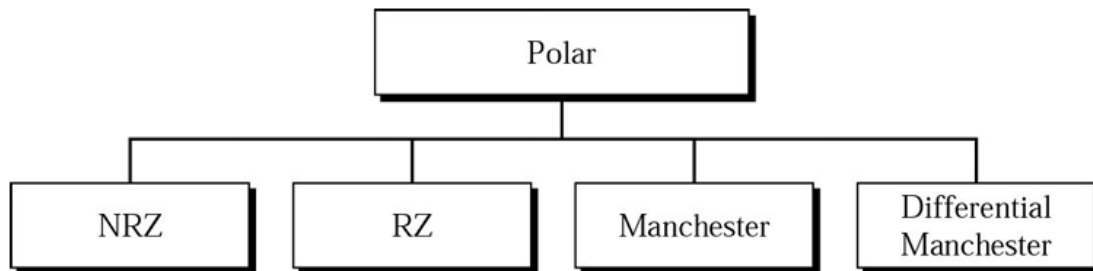
◆ Disadvantages

- ✗ Has DC component → cannot pass through AC-coupled links.
- ✗ Poor synchronization (long runs of 0s or 1s cause errors).
- ✗ Higher average power consumption than other schemes

✓ In summary:

Unipolar NRZ is the simplest coding scheme where 1 = +V and 0 = 0 V.

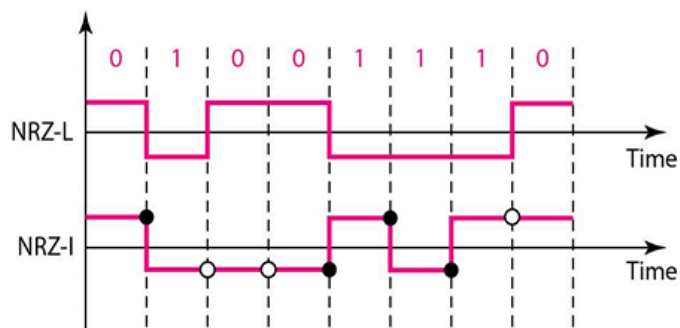
It is easy to implement but suffers from **DC component** and **synchronization problems**, so it is rarely used in modern digital communication.



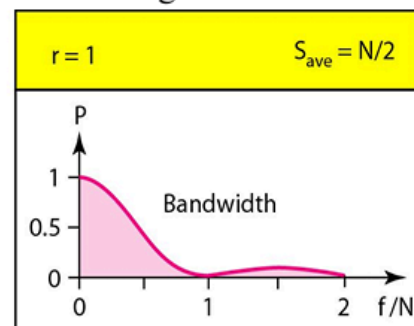
NRZ-L and NRZ-I

- ❑ In Non-return to Zero-level (NRZ-L) the level of the signal is dependent upon the state of the bit.
- ❑ In Non-return to Zero-Invert (NRZ-I) the signal is inverted if a 1 is encountered.

Figure 4.6 Polar NRZ-L and NRZ-I schemes



r: average baud rate
s: signal rate



NRZ-L AND NRZ-I (POLAR LINE CODING)

◆ What They Are

1. NRZ-L (Non-Return-to-Zero Level)

- The **voltage level directly represents the bit**.
 - Bit 1 → one voltage (e.g., +V).
 - Bit 0 → opposite voltage (e.g., -V).
- The signal level stays constant during the bit interval → it does **not return to zero**.

👉 Rule: **Voltage level = Bit value**.

2. NRZ-I (Non-Return-to-Zero Invert)

- The signal **changes (inverts)** if the bit is 1.
- The signal **remains the same** if the bit is 0.

👉 Rule: **Inversion = 1, No inversion = 0**.

- So information is carried in **transitions**, not in absolute voltage levels.
-

◆ Figure Explanation

- The diagram shows the same data sequence 01011110.
- **NRZ-L**: Each bit's value directly sets the voltage.
- **NRZ-I**: A 1 triggers a flip in voltage; a 0 leaves it unchanged.

Markers in figure:

- White circle = no inversion (bit = 0).
 - Black circle = inversion (bit = 1).
-

◆ Bandwidth and Baud Rate

- For both NRZ-L and NRZ-I:
 - $r = 1$ (1 bit per 1 signal element).
 - Average signal rate: $S_{ave} = N/2S_{\{ave\}} = N/2$ baud.
 - Bandwidth requirement is about **half the bit rate**.
-

◆ Advantages & Disadvantages

NRZ-L

- ✓ Simple implementation.
- ✗ Long runs of 0s or 1s cause **loss of synchronization** (no transitions).
- ✗ Has a **DC component** (average signal not centered at zero).

NRZ-I

- ✓ Better synchronization than NRZ-L (long runs of 1s still produce transitions).
 - ✓ More reliable in practice (transition-based).
 - ✗ Still suffers from DC component.
-

◆ Key Difference

- **NRZ-L:** Bit value determined by **voltage level**.
 - **NRZ-I:** Bit value determined by **change (inversion)** or no change in voltage.
-

✓ In summary:

- Both are **polar coding schemes** using +V and -V.

- **NRZ-L** → “Level = Bit value.”
- **NRZ-I** → “Inversion = 1, No inversion = 0.”
- Average baud rate: $N/2$.
- Both face DC and sync issues, but **NRZ-I is usually preferred** because transitions make it easier to keep clocks aligned.

👉 Do you want me to also prepare a **comparison table of Unipolar NRZ vs NRZ-L vs NRZ-I** so you can see their differences side by side for exams?

NRZ-L and NRZ-I both have an average signal rate of $N/2$ Bd.

NRZ-L and NRZ-I both have a DC component problem.

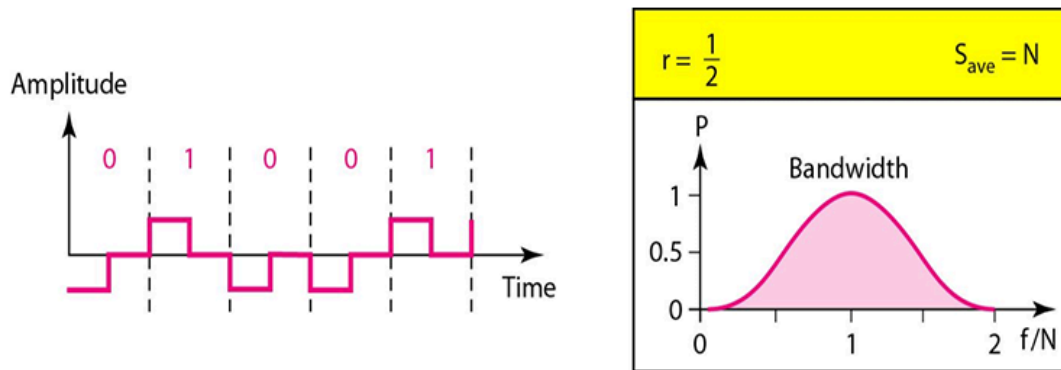
A system is using NRZ-I to transfer 1 Mbps data. What are the average signal rate and minimum bandwidth?

Solution

The average signal rate is $S = N/2 = 500$ kbaud. The minimum bandwidth for this average baud rate is $B_{\min} = S = 500$ kHz.

RZ (Return-to-zero)

Figure 4.7 Polar RZ scheme



A good encoded digital signal must contain a provision for synchronization.

RZ (RETURN-TO-ZERO) LINE CODING

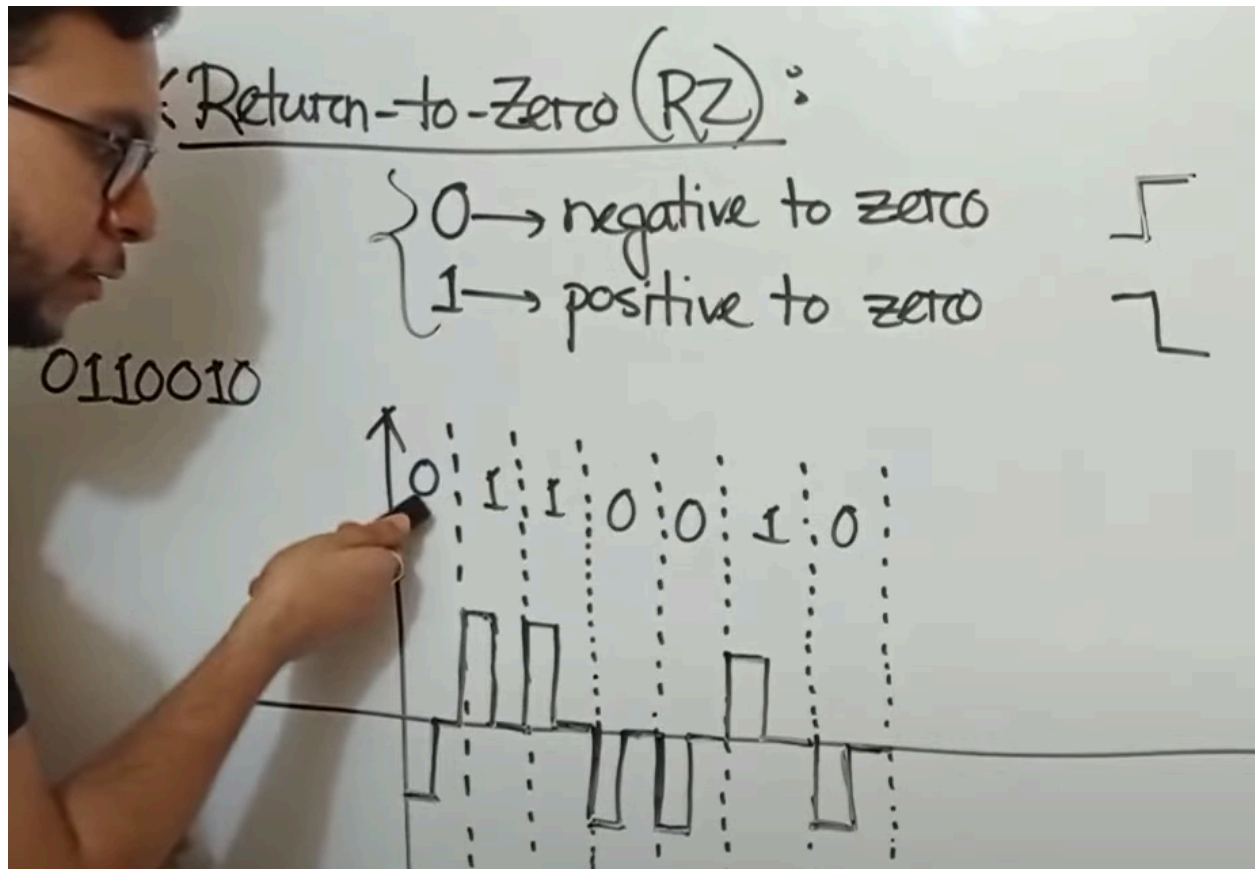
◆ What It Is

- RZ (Return-to-Zero) is a polar line coding scheme.
- Each bit interval is split into **two halves**:
 - In the **first half**, the voltage represents the bit value.
 - In the **second half**, the signal returns to zero (baseline).

👉 That's why it's called **Return-to-Zero (RZ)**.

0 means Negative to Zero

1 means Positive to 0



◆ Waveform (Figure Explanation)

- For a bit 1 → signal goes positive (e.g., +V) during the first half, then returns to 0.
- For a bit 0 → signal goes negative (e.g., -V) during the first half, then returns to 0.
- So in every bit interval, there is **always a transition back to zero**, regardless of the next bit.

This makes **synchronization easier** because there are frequent transitions.

◆ Bandwidth and Signal Rate

- In RZ: 1 bit → 2 signal elements (because of the “go to zero” rule).
- So $r = \frac{1}{2}$.
- Signal rate:

$$S_{ave} = N$$

where N is the bit rate.

👉 That means the baud rate = bit rate (unlike NRZ where $S = N/2$).

- Bandwidth requirement is higher than NRZ.

◆ Characteristics

- **Self-Synchronization:** Very good (since there is always a transition).
- **DC Component:** Reduced compared to NRZ, because the signal often returns to 0.
- **Bandwidth:** Higher than NRZ, since each bit produces more transitions.

◆ Advantages

- ✓ Good synchronization (every bit has a transition).
- ✓ Easier to detect bits in noisy environments.

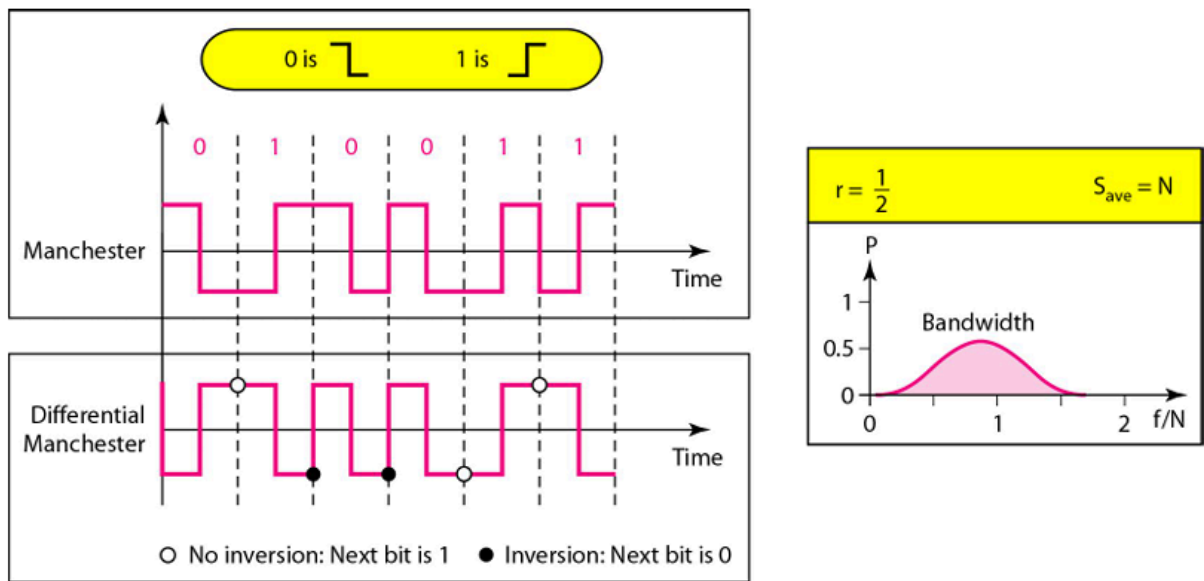
◆ Disadvantages

- ✗ Requires **higher bandwidth** than NRZ.
 - ✗ More complex circuitry.
 - ✗ Not power efficient (lots of transitions).
-

✓ Summary

- **RZ** → Each bit returns to zero mid-bit.
- $r = \frac{1}{2}$, $S = N$ (baud).
- Provides good synchronization, but at the cost of **more bandwidth** than NRZ.

Figure 4.8 *Polar biphas: Manchester and differential Manchester schemes*



BIPHASE CODING: MANCHESTER & DIFFERENTIAL MANCHESTER

◆ What It Is

- Biphase coding is a **polar line coding scheme** that guarantees **at least one transition per bit**, making it **self-synchronizing**.

- Two common variants:
 1. **Manchester Encoding**
 2. **Differential Manchester Encoding**

Both are widely used in LAN technologies (e.g., **Ethernet**).

♦ 1. Manchester Encoding

- **Rule:**
 - 0 → High-to-Low transition in the middle of the bit.
 - 1 → Low-to-High transition in the middle of the bit.
- The **mid-bit transition** serves two purposes:
 - Carries the bit value.
 - Provides **synchronization** (receiver can lock onto the clock).
- Each bit requires **two signal elements** → $r = 1/2r = 1/2$.
- **Signal rate = N (baud).**

👉 Example in figure:

Bit sequence 0100111 produces transitions in the middle of each bit period.

♦ 2. Differential Manchester Encoding

- **Rule:**
 - Always has a **transition in the middle** (for synchronization).
 - A 0 → **transition at the beginning** of the bit period.

- A 1 → no transition at the beginning.
- Advantage:
 - Data is encoded based on **transitions** rather than absolute voltage.
 - Makes it more robust against polarity inversions (e.g., wire reversal).

👉 In the figure:

- Hollow circle ○ = No inversion (next bit = 1).
- Filled circle ● = Inversion (next bit = 0).

◆ Bandwidth and Signal Rate

- For both Manchester and Differential Manchester:
 - $r = \frac{1}{2}$ (2 signal elements per bit).
 - Average signal rate:

$$S_{ave} = N$$

where N = bit rate.

- Bandwidth is about double NRZ schemes.

◆ Advantages

- ✓ Excellent synchronization (mid-bit transition guarantees timing).
- ✓ No DC component (balanced positive/negative levels).
- ✓ Differential Manchester is immune to polarity reversal.

◆ Disadvantages

- ✗ Requires **more bandwidth** (2× compared to NRZ).
 - ✗ More complex to implement.
-

✓ Summary

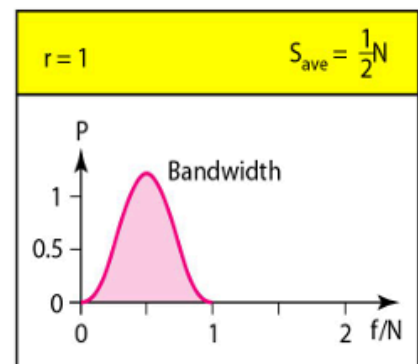
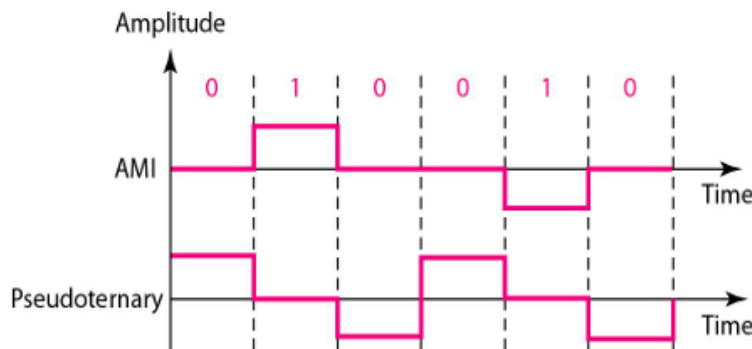
- **Manchester:** Transition in middle defines the bit (0 = high→low, 1 = low→high).
- **Differential Manchester:** Mid-bit transition always present; beginning transition defines the bit.
- Both → $S=NS = N$, need more bandwidth but provide **strong self-synchronization**.

Bipolar Schemes

□ In bipolar encoding, we use three levels:

- positive, zero, and negative.

□ ***Bipolar schemes: AMI and pseudoternary***



BIPOLAR SCHEMES (AMI & PSEUDOTERNARY)

◆ What It Is

- **Bipolar encoding** is a line coding scheme that uses **three voltage levels**:
 - Positive (+V)
 - Zero (0)
 - Negative (−V)
 - Unlike **unipolar** (only 0 and +V) or **polar** (−V and +V), bipolar has an extra level (0), which reduces **DC component** and helps in **synchronization**.
-

◆ Two Main Schemes

1. AMI (Alternate Mark Inversion)

- **Rule:**
 - Bit 0 → represented by **0 V** (no signal).
 - Bit 1 → represented by alternating +V and −V levels.
- **Example:**
 - First 1 → +V
 - Second 1 → −V
 - Third 1 → +V (alternates again)

- This ensures there is **no DC component** because positive and negative pulses balance out.
-

2. Pseudoternary

- The **opposite of AMI**.
 - **Rule:**
 - Bit 1 $\rightarrow$ 0 V
 - Bit 0 $\rightarrow$ represented by alternating +V and -V
 - Used in some systems as an alternative to AMI.
-

◆ Figure Explanation

- The diagram shows:
 - **AMI:** Only 1s are alternating between +V and -V, while 0s are 0 V.
 - **Pseudoternary:** Only 0s alternate between +V and -V, while 1s are 0 V.

Both ensure a balance of positive and negative signals.

◆ Bandwidth and Signal Rate

- $r = 1 \rightarrow 1$ data bit per signal element.
- Average signal rate:

$$S_{ave} = \frac{N}{2}$$

where N is the bit rate.

- Bandwidth requirement is similar to NRZ but with better synchronization and DC suppression.

◆ Advantage

- ✓ No DC component (balanced +V and -V).
 - ✓ Better synchronization (frequent transitions when 1s or 0s alternate).
 - ✓ Power efficient (average power lower than unipolar).
-

◆ Disadvantages

- ✗ Still not guaranteed transitions if a long run of 0s (for AMI) or 1s (for pseudoternary).
 - ✗ Requires scrambling or extra coding to avoid long runs of constant values.
-

✓ Summary

- Bipolar = three levels (+V, 0, -V).
- AMI: 0 = 0V, 1 = alternating +V/-V.
- Pseudoternary: 1 = 0V, 0 = alternating +V/-V.
- Signal rate: $S = N/2S = N/2$.
- Benefit: Low DC, better sync, less power than unipolar.

👉 Do you want me to also compare **Unipolar vs Polar vs Bipolar** in a single **table format** so you can memorize the differences quickly for exams?